
BY COMMUNITY • FOR COMMUNITY •
#GLOBALAZURE
TORINO



Azure Meetup
TORINO

alveo®
EXPERTISE

TD SYNNEX

#GlobalAzureTorino

Azure Kubernetes Services multi-region setup and disaster recovery

Rauno De Pasquale

Azure Meetup TORINO

alveo EXPERTISE

TD SYNNEX



About me



Rauno
De Pasquale

- **Twitter**: @RaunoDepa
- **GitHub**: https://github.com/raunodepasquale
- **SlideShare**: https://www.slideshare.net/RaunoDePasquale
- **LinkedIn**: https://www.linkedin.com/in/rauno-de-pasquale-b075773/
- **Medium**: https://medium.com/@rauno.depasquale

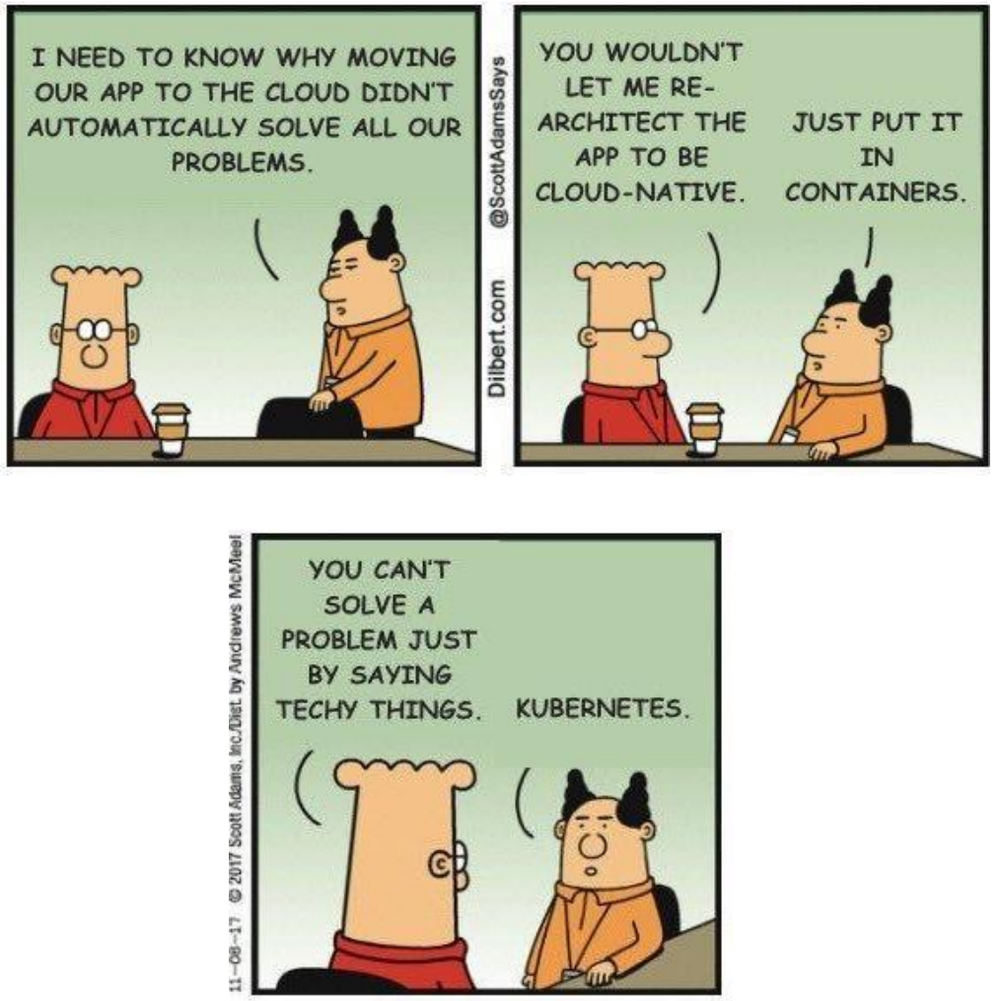
Azure Meetup TORINO

alveo EXPERTISE

TD SYNNEX

Agenda

- The Context
- The Problem
- The Solution



#GlobalAzureTorino

The Context

Application Architecture and Business Requirements

Azure Meetup TORINO

alveo EXPERTISE

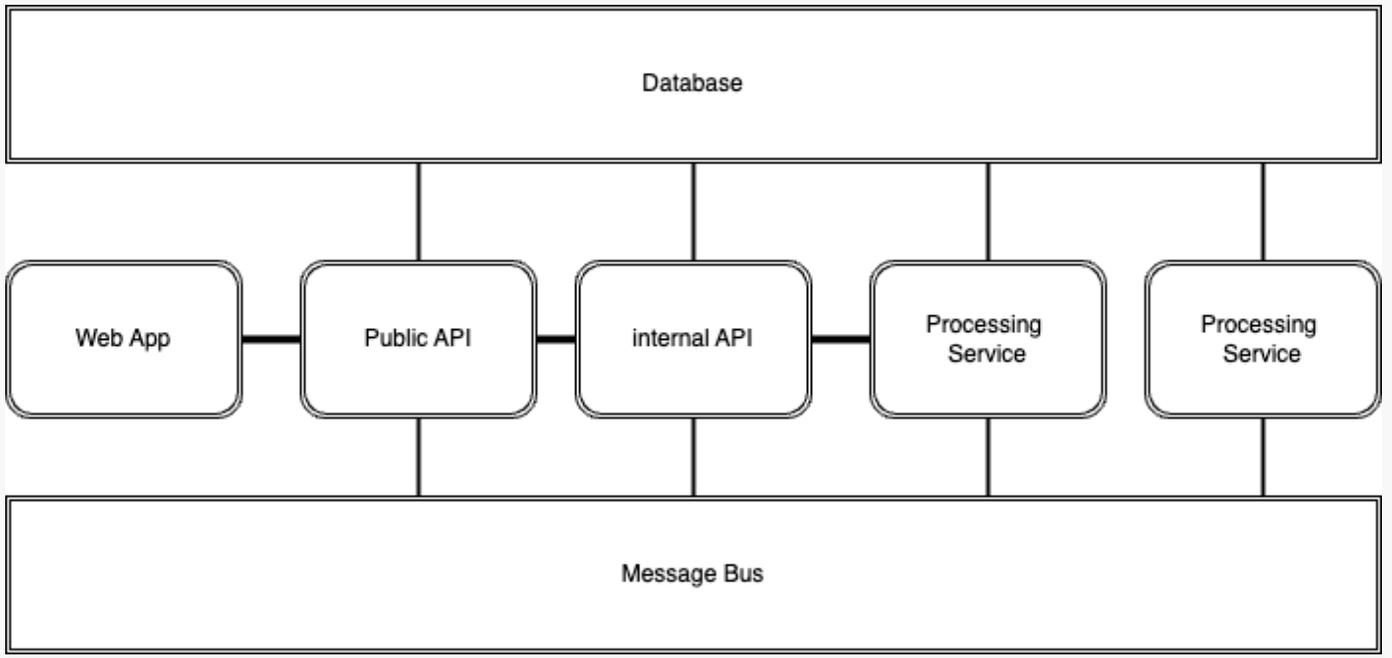
TD SYNNEX

#GlobalAzureTorino

High Level Architecture

Micro Services Architecture

- Multiple components
- Containerized Software
- Publicly exposed HTTPS Endpoints
- Internally exposed HTTPS Endpoints
- Unattended Services
- AMQP Messaging



#GlobalAzureTorino

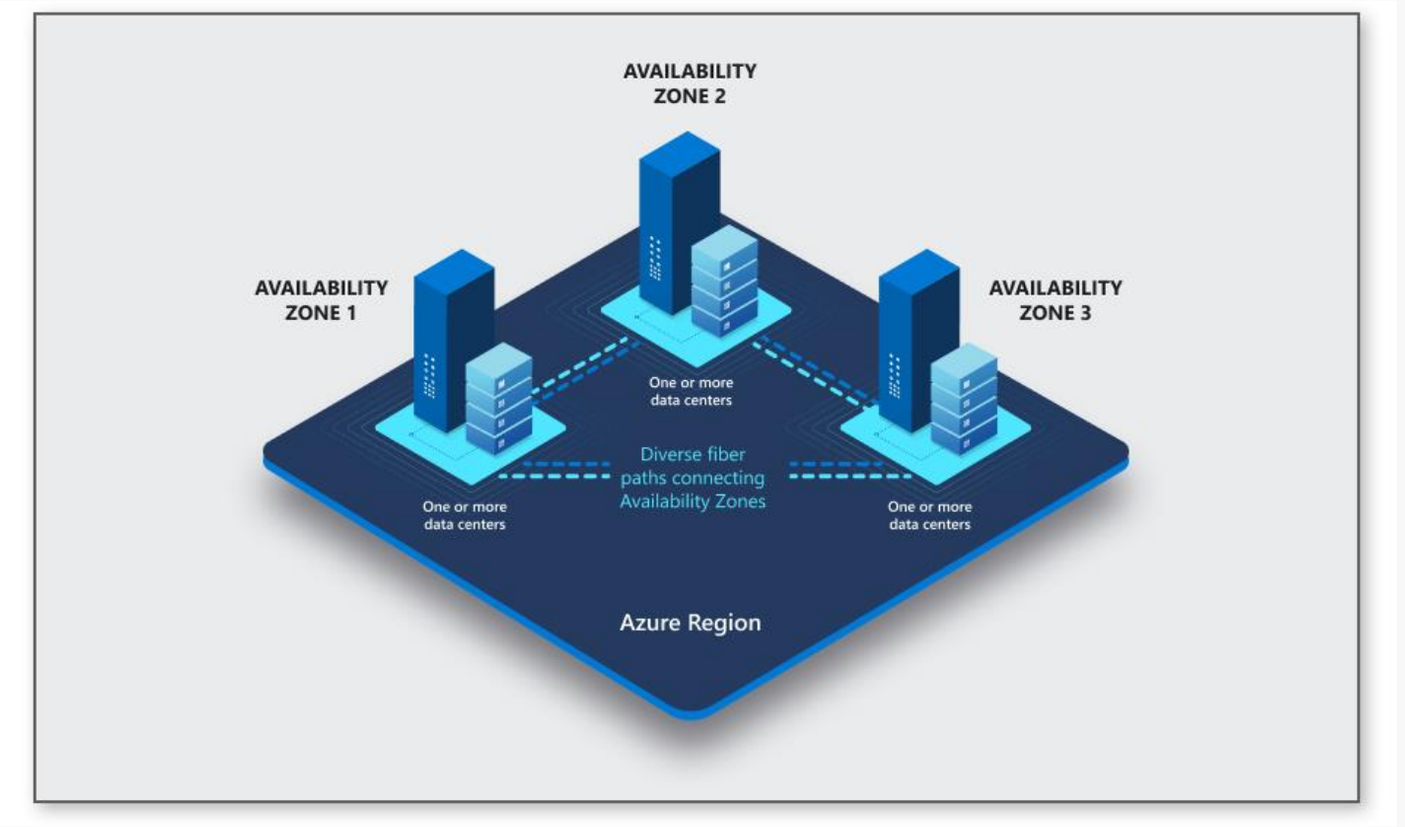
Business Context

High availability

99.95% availability not enough

Protection from regional disaster

Mitigate cost impacts



#GlobalAzureTorino

The Problem

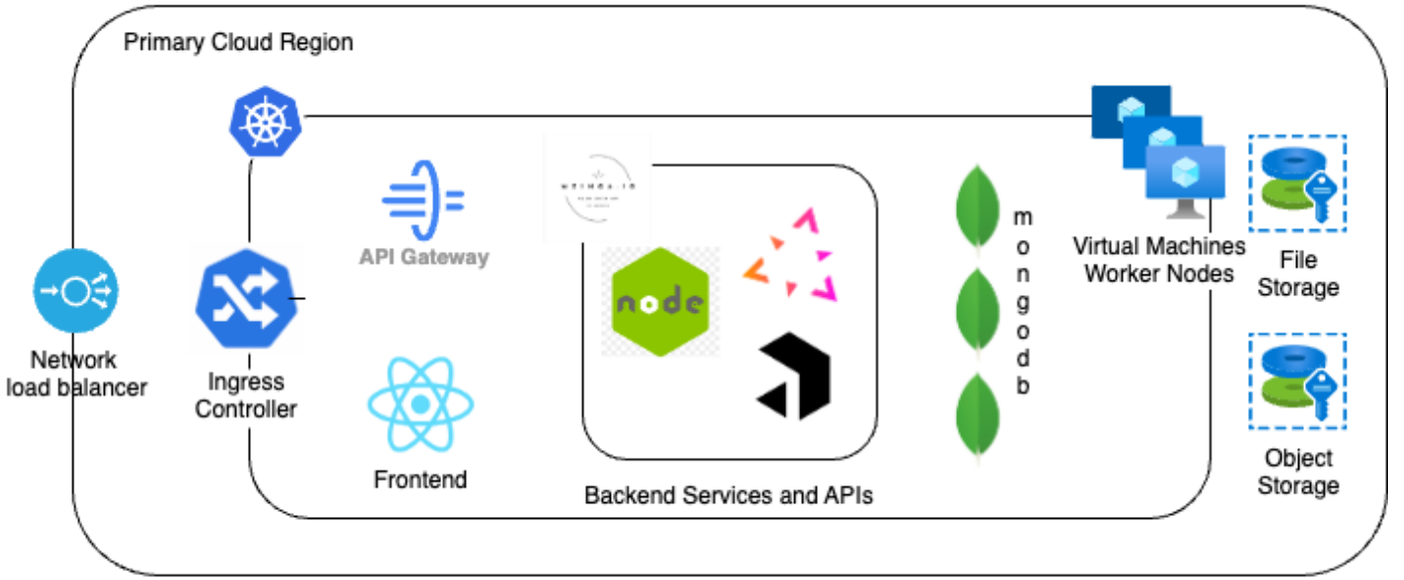
Business Continuity vs Cost Management

Azure Meetup TORINO

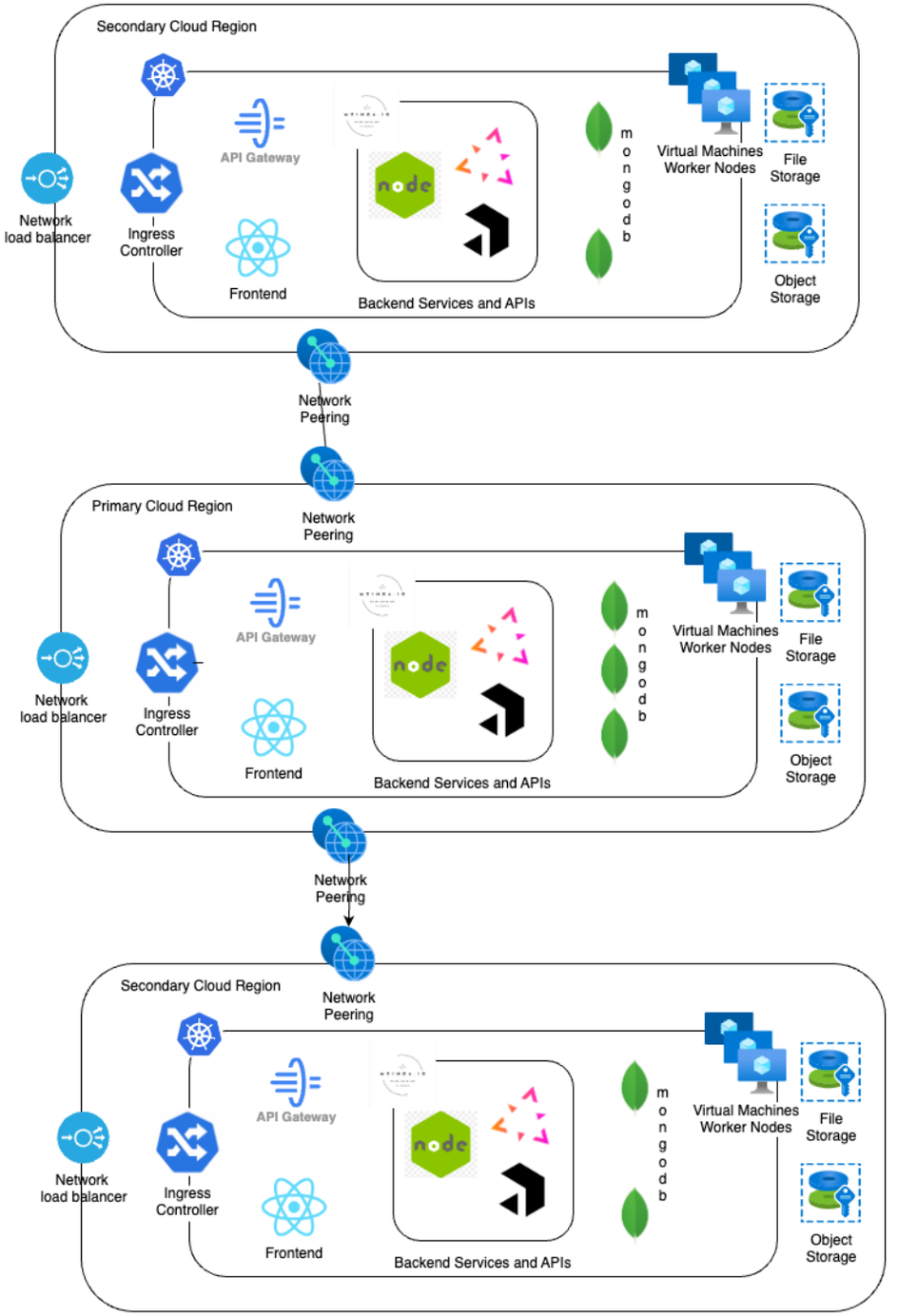
alveo EXPERTISE

TD SYNNEX

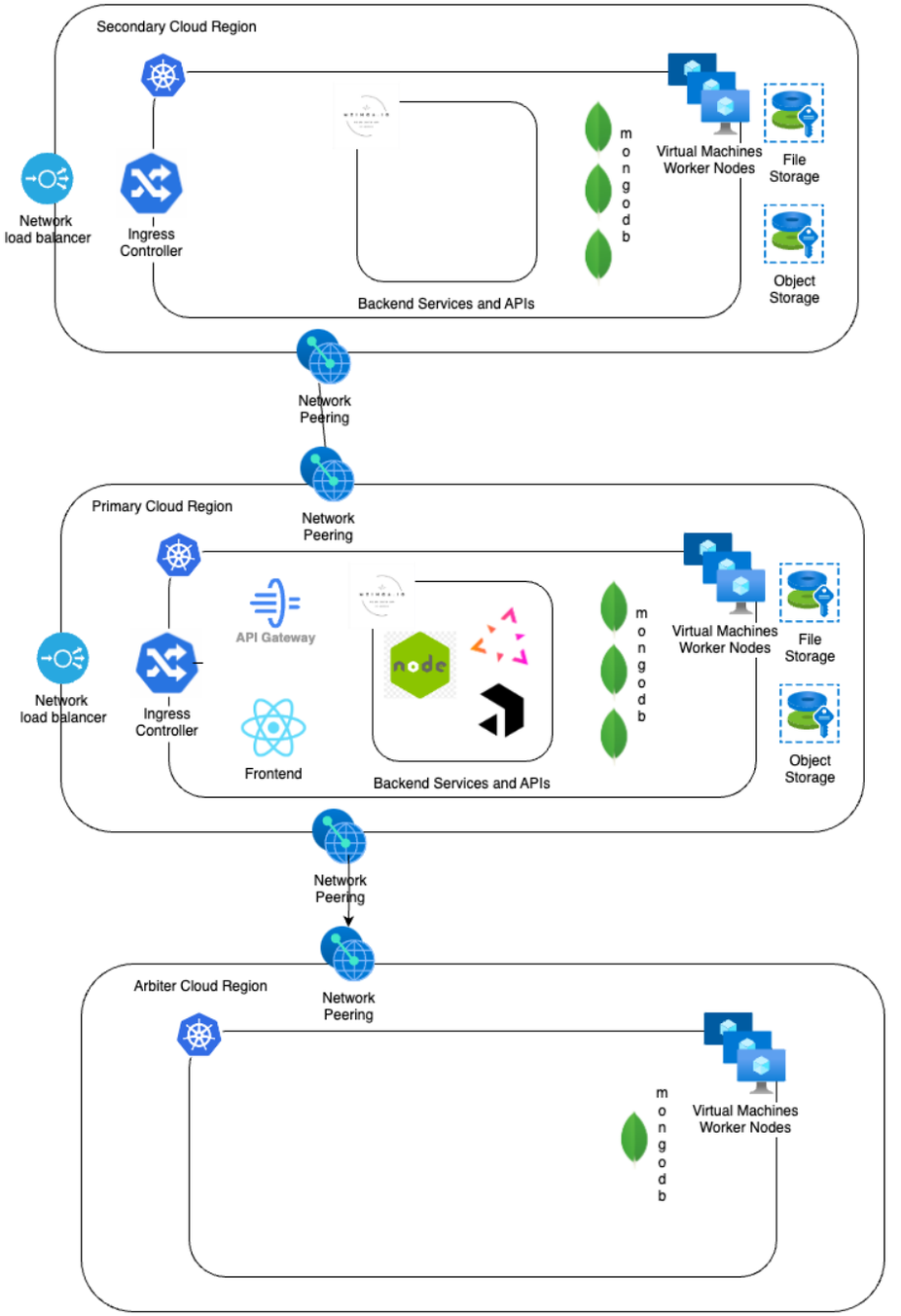
Single Region Design Geo Distributed Backups



Multi Region Design Active / Active Scenario



Multi Region Design Active / Passive Scenario



#GlobalAzureTorino

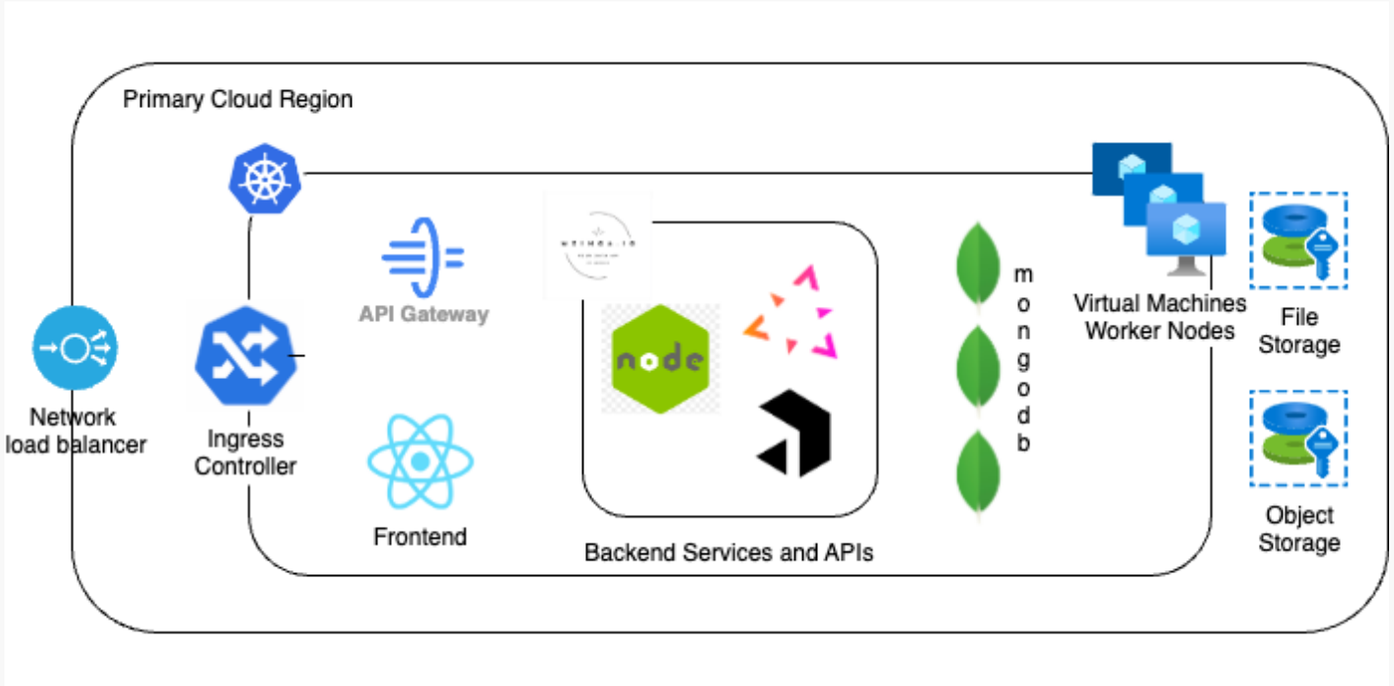
Multi Region Setup

Problems

How to route traffic to the right location?

Horizontal Pod Autoscaler is usually based on CPU or Memory

Horizontal Pod Autoscaler requires a minimum of 1 replica



#GlobalAzureTorino

The Solution

Event Driven Scaling Rules

Azure Meetup TORINO

alveo EXPERTISE

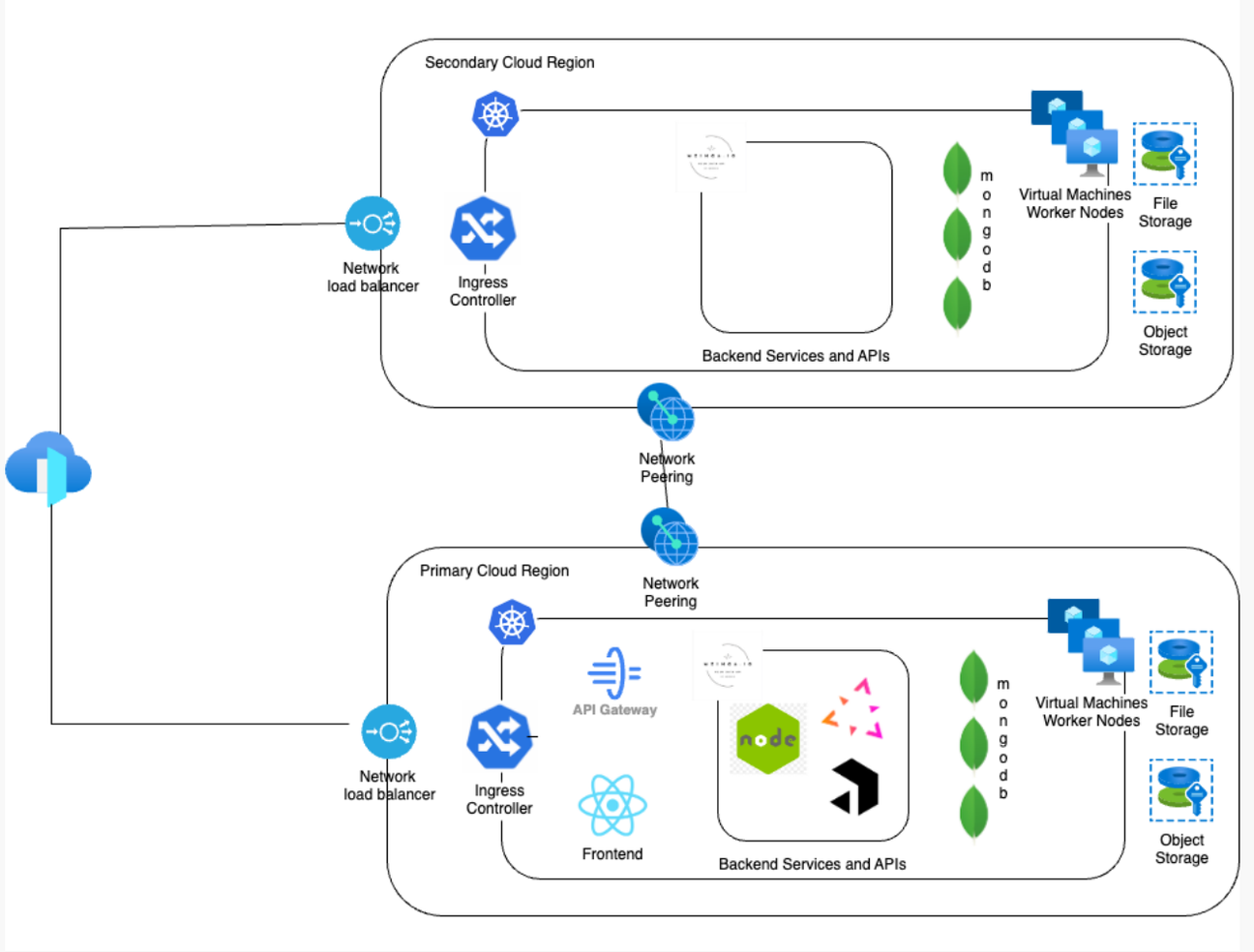
TD SYNNEX

#GlobalAzureTorino

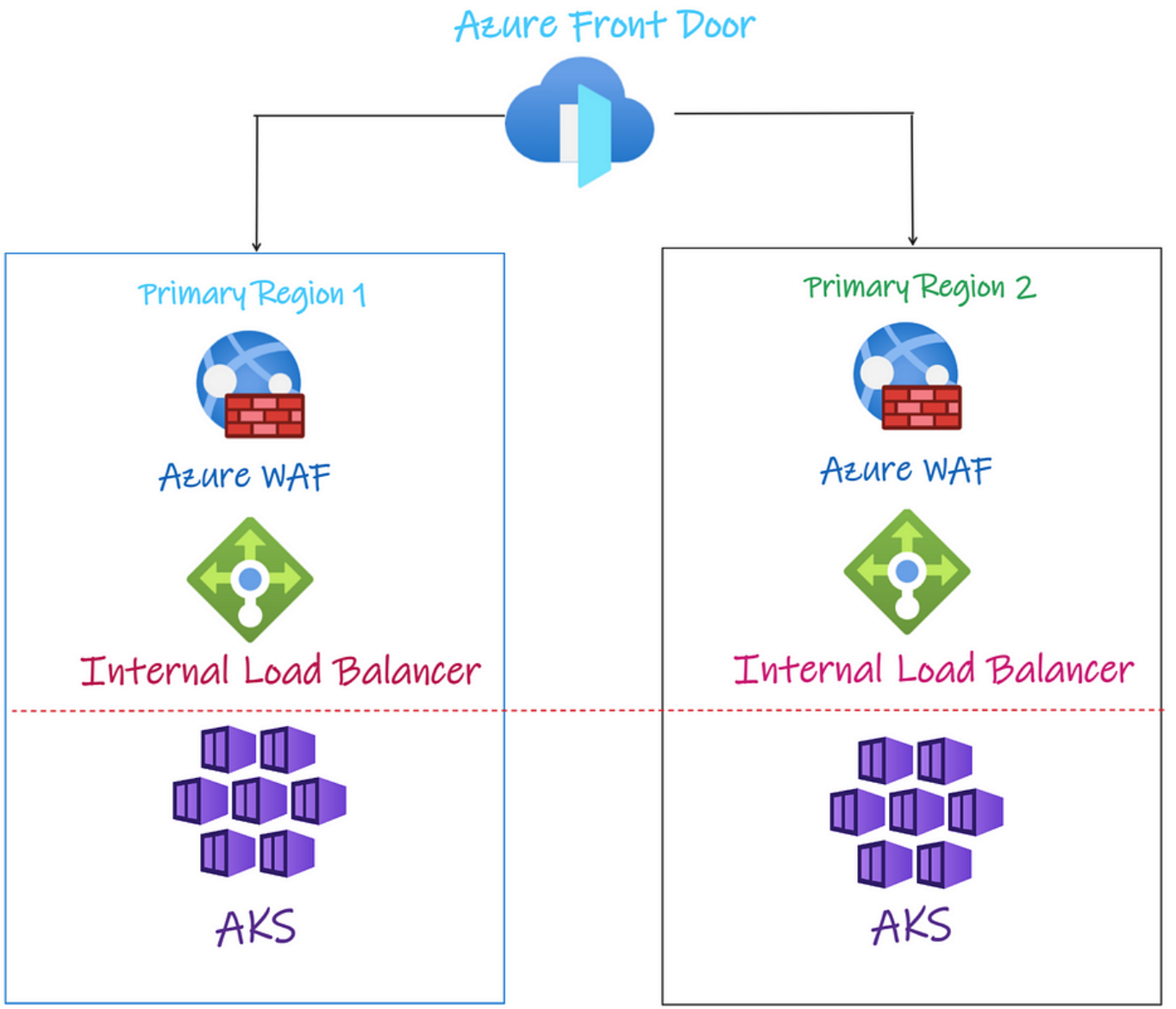
Multi Region Setup

Solution

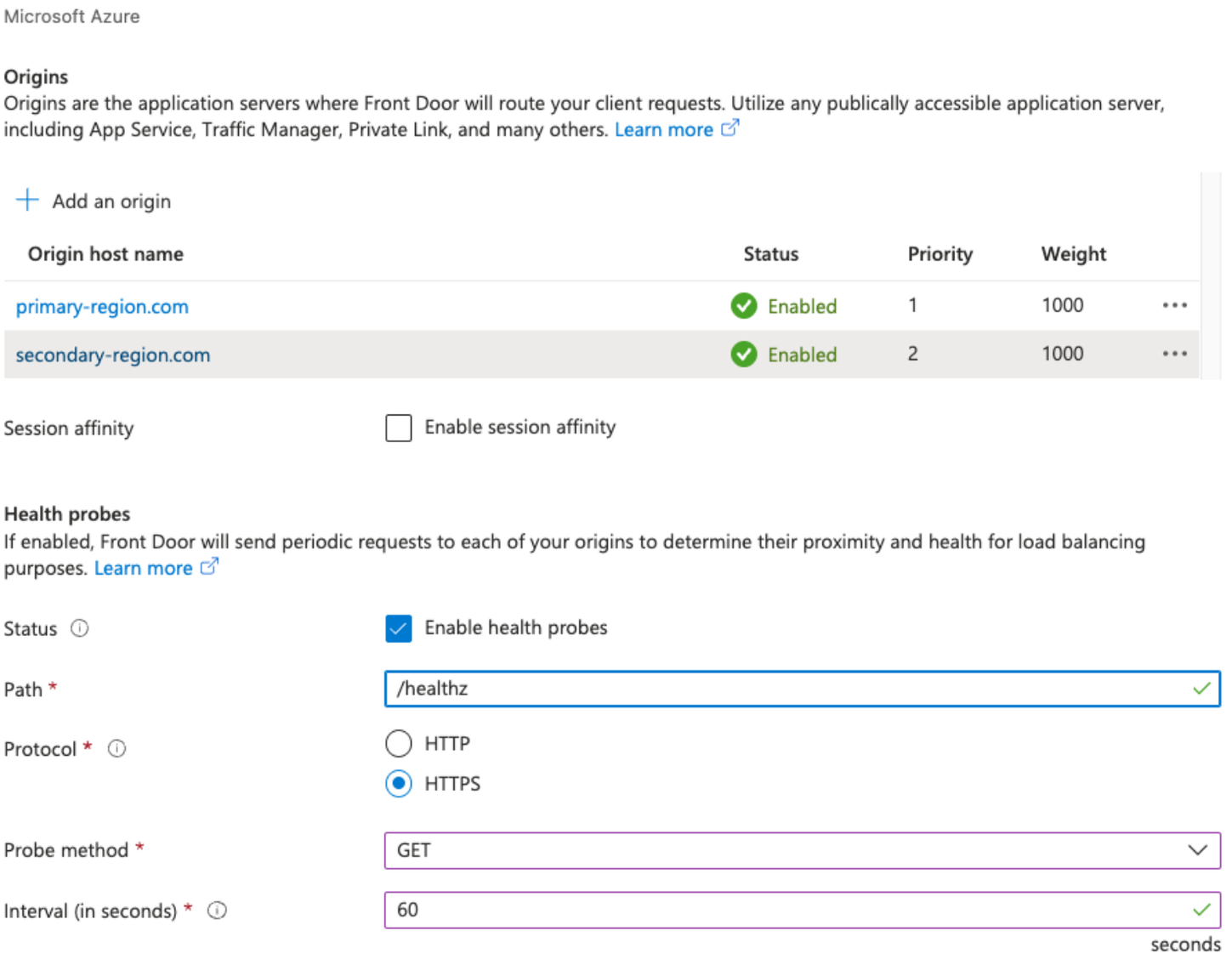
- Azure FrontDoor CDN
- Vertical Pod Autoscaling
- KEDA to control Horizontal Pod Autoscaling
- Taint, Affinity and Toleration to drive resource allocations to nodes



Multi Region Traffic Routing



Multi Region Traffic Routing



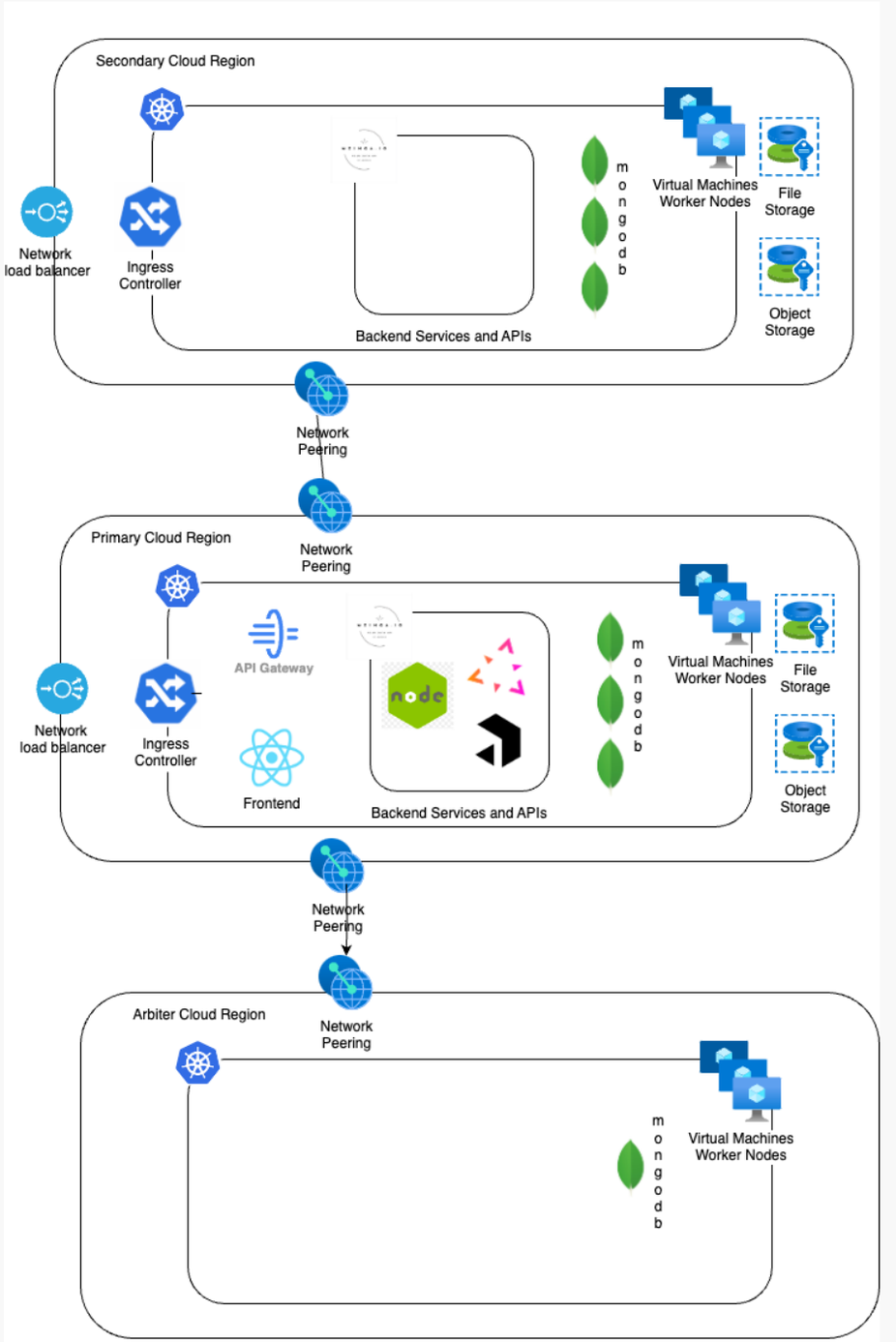
#GlobalAzureTorino

Multi Region Setup

Data Management

Usage of a witness region to assure majority rules on MongoDb replicaset

Percona Operator to manage replicaset and backup with point in time restore

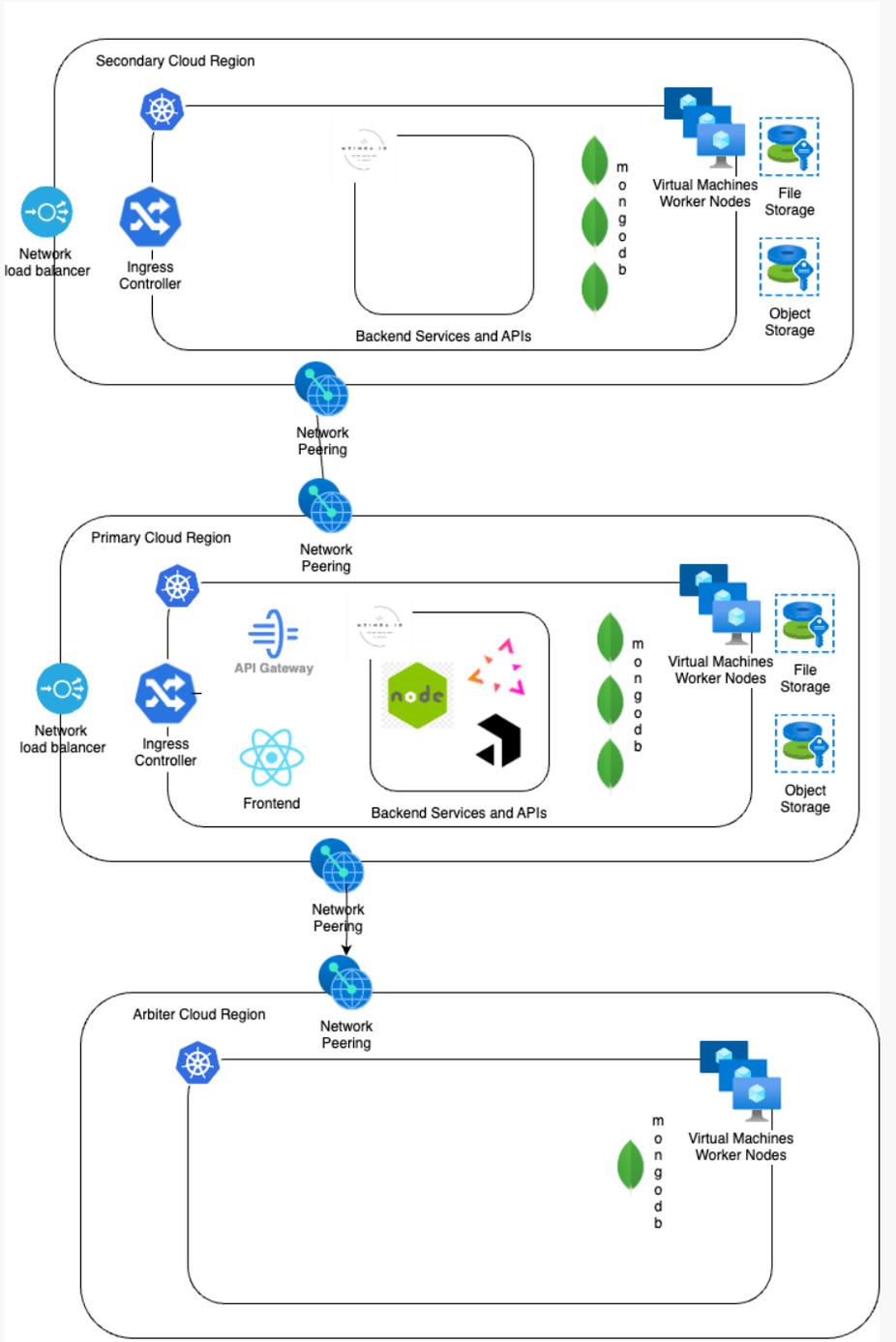


#GlobalAzureTorino

Multi Region Setup

Application Management

- Application Deployment done using ArgoCD
- Deployments are always coherent in the two clusters
- Application scaling settings are different using regional value files to override default values



Multi Region Application Deployment

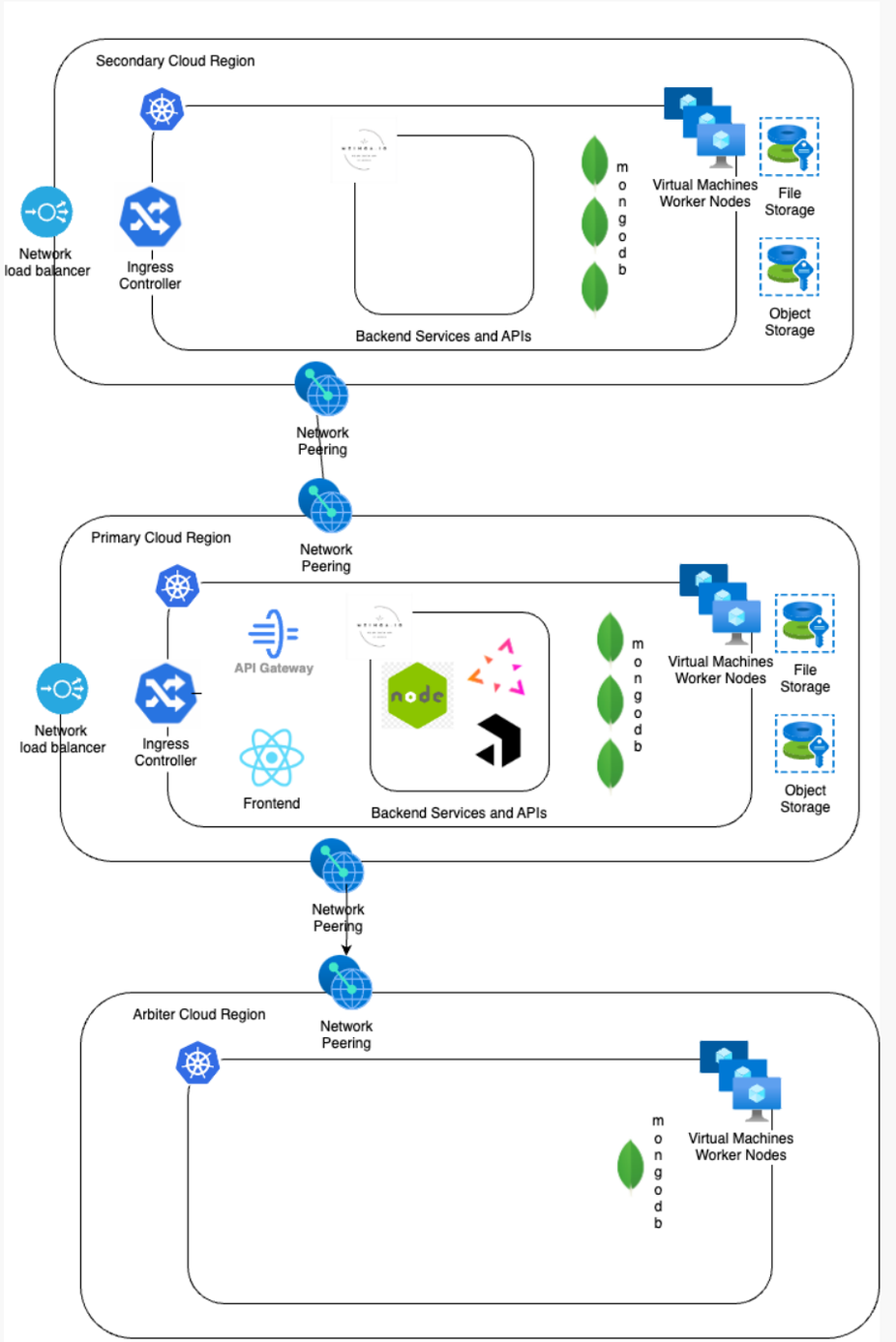
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  annotations:
    argocd.argoproj.io/manifest-generate-paths: .
  finalizers:
    - resources-finalizer.argocd.argoproj.io
  labels:
    app: rabbitmq-cluster
    env: prod
    org: publishing
    region: primary
    team: mzinga
  name: publishingcms-rabbitmq-cluster-mzinga-prod-primary
  namespace: argocd
spec:
  destination:
    name: mzinga-aks-publishingcms-primary
    namespace: mzinga-app
  project: mzinga-publishing
  source:
    helm:
      releaseName: rabbitmq-cluster
      valueFiles:
        - values.yaml
        - env/prod/primary/mzinga/values.yaml
        - env/prod/primary/mzinga/secrets.yaml
      version: v3
    path: helm/extensions/rabbitmq-clusters/
    repoURL: https://github.repo.url/mzinga/rabbitmq
    targetRevision: main
  syncPolicy:
    automated:
      prune: true
    syncOptions:
      - ApplyOutOfSyncOnly=true
      - CreateNamespace=true
      - PruneLast=true
      - PrunePropagationPolicy=background
      - RespectIgnoreDifferences=true
```

#GlobalAzureTorino

Multi Region Setup

Cluster Resource Scaling

- Multiple Node Pools
- Autoscaling starting from zero
- Taint to define applications allocations to nodes
- Multizone pools with multizone disks



Multi Region Cluster Resource Scaling

```hcl
resource "azurerm_kubernetes_cluster" "aks" {
  name                = "aks-cluster"
  location            = var.location
  resource_group_name = var.resource_group_name
  dns_prefix          = "aks"

  default_node_pool {
    name                = "system"
    vm_size             = "Standard_B2ms" # 2 vCPU, 8 GB RAM
    node_count          = 3
    min_count           = 3
    max_count           = 9
    enable_auto_scaling = true
    mode                = "System"
    availability_zones  = ["1", "2", "3"]
    node_labels = {
      approle = "core"
    }
  }

  identity {
    type = "SystemAssigned"
  }

  kubernetes_version = "1.31.7"
  role_based_access_control {
    enabled = true
  }

  network_profile {
    network_plugin = "azure"
    load_balancer_sku = "standard"
  }
}
```

Multi Region Cluster Resource Scaling

```
resource "azurerm_kubernetes_cluster_node_pool" "application" {
  name                  = "application"
  kubernetes_cluster_id = azurerm_kubernetes_cluster.aks.id
  vm_size               = "Standard_D4s_v5" # 4 vCPU, 8 GB RAM
  min_count             = 0
  max_count             = 9
  enable_auto_scaling   = true
  mode                  = "User"
  availability_zones    = ["1", "2", "3"]
  orchestrator_version  = azurerm_kubernetes_cluster.aks.kubernetes_version
  node_labels = {
    approle = "apps"
  }
  node_taints = [
    "approle=apps:NoSchedule"
  ]
}

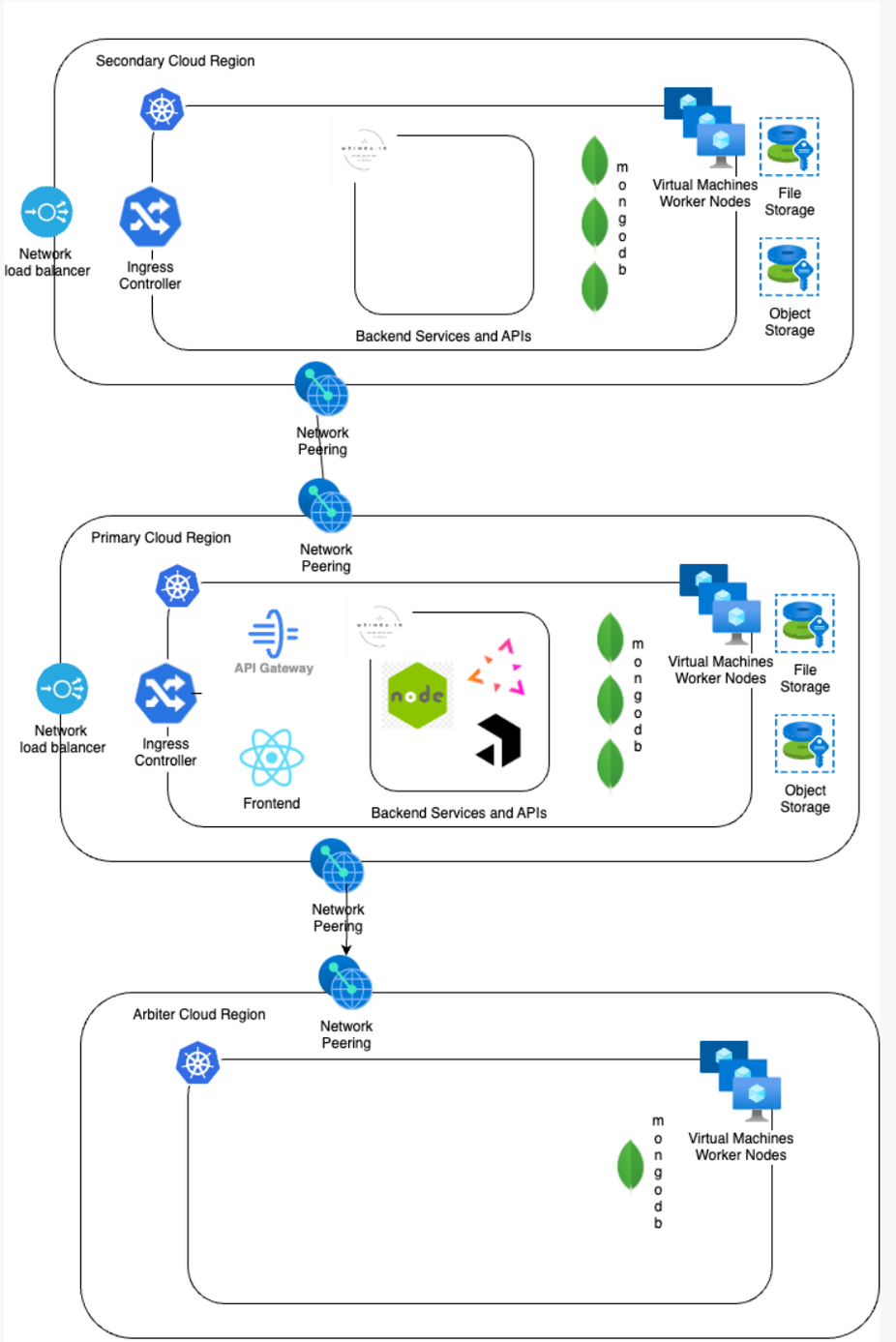
resource "azurerm_kubernetes_cluster_node_pool" "database" {
  name                  = "database"
  kubernetes_cluster_id = azurerm_kubernetes_cluster.aks.id
  vm_size               = "Standard_D16s_v5" # 16 vCPU, 32 GB RAM
  min_count             = 0
  max_count             = 3
  enable_auto_scaling   = true
  mode                  = "User"
  availability_zones    = ["1", "2", "3"]
  orchestrator_version  = azurerm_kubernetes_cluster.aks.kubernetes_version
  node_labels = {
    approle = "database"
  }
  node_taints = [
    "approle=database:NoSchedule"
  ]
}
```

#GlobalAzureTorino

Multi Region Setup

Application Scaling

- Keda RabbitMQ Integration
- Keda HTTP Add On
- Vertical Pod Autoscaling
- Pod Disruption Budget



Multi Region Application Scaling

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mzinga-core
  labels:
    app: mzinga-core
spec:
  replicas: 0
  selector:
    matchLabels:
      app: mzinga-core
  template:
    metadata:
      labels:
        app: mzinga-core
    spec:
      tolerations:
      - key: "approle"
        operator: "Equal"
        value: "apps"
        effect: "NoSchedule"
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - key: "kubernetes.azure.com/agentpool"
                operator: In
                values:
                - application
      containers:
      - name: mzinga-core
        image: your-registry/mzinga-core:latest
        env:
        - name: RABBITMQ_CONN_STR
          valueFrom:
            secretKeyRef:
              name: rabbitmq-secret
              key: RABBITMQ_CONN_STR
```

Multi Region Application Scaling

```yaml
kind: HTTPScaledObject
apiVersion: http.keda.sh/v1alpha1
metadata:
    name: mzinga-api
    annotations:
        httpscaledobject.keda.sh/skip-scaledobject-creation: "false"
spec:
    hosts:
    - api.mzinga.io
    pathPrefixes:
    - /test
    scaleTargetRef:
        name: mzinga-api
        kind: Deployment
        apiVersion: apps/v1
        service: mzinga-api
        port: 8080
    replicas:
        min: 0
        max: 10
    scaledownPeriod: 300
    scalingMetric: # requestRate and concurrency are mutually exclusive
        requestRate:
            granularity: 1s
            targetValue: 100
            window: 1m
```

Multi Region Application Scaling

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: mzinga-core-scaler
spec:
  scaleTargetRef:
    name: mzinga-core
  minReplicaCount: 0
  maxReplicaCount: 5
  triggers:
  - type: rabbitmq
    metadata:
      queueName: core-triggers
      hostFromEnv: RABBITMQ_CONN_STR
      mode: QueueLength
      value: "5"
```

Multi Region Application Scaling

```yaml
# Tolerate taint from database node pool
tolerations:
- key: "approle"
  operator: "Equal"
  value: "database"
  effect: "NoSchedule"
# Prefer nodes with approle=database, but not required
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 100
      preference:
        matchExpressions:
        - key: "approle"
          operator: In
          values:
          - database
```

Multi Region Application Scaling

```yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: mongodb-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: StatefulSet
    name: mongodb
  updatePolicy:
    updateMode: "Auto"
  resourcePolicy:
    containerPolicies:
    - containerName: "*"
      controlledResources: ["cpu", "memory"]
      controlledValues: "RequestsAndLimits"
      minAllowed:
        cpu: "500m"
        memory: "1Gi"
      maxAllowed:
        cpu: "12"
        memory: "32Gi"
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: mongodb-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: mongodb
```

#GlobalAzureTorino

Multi Region Setup

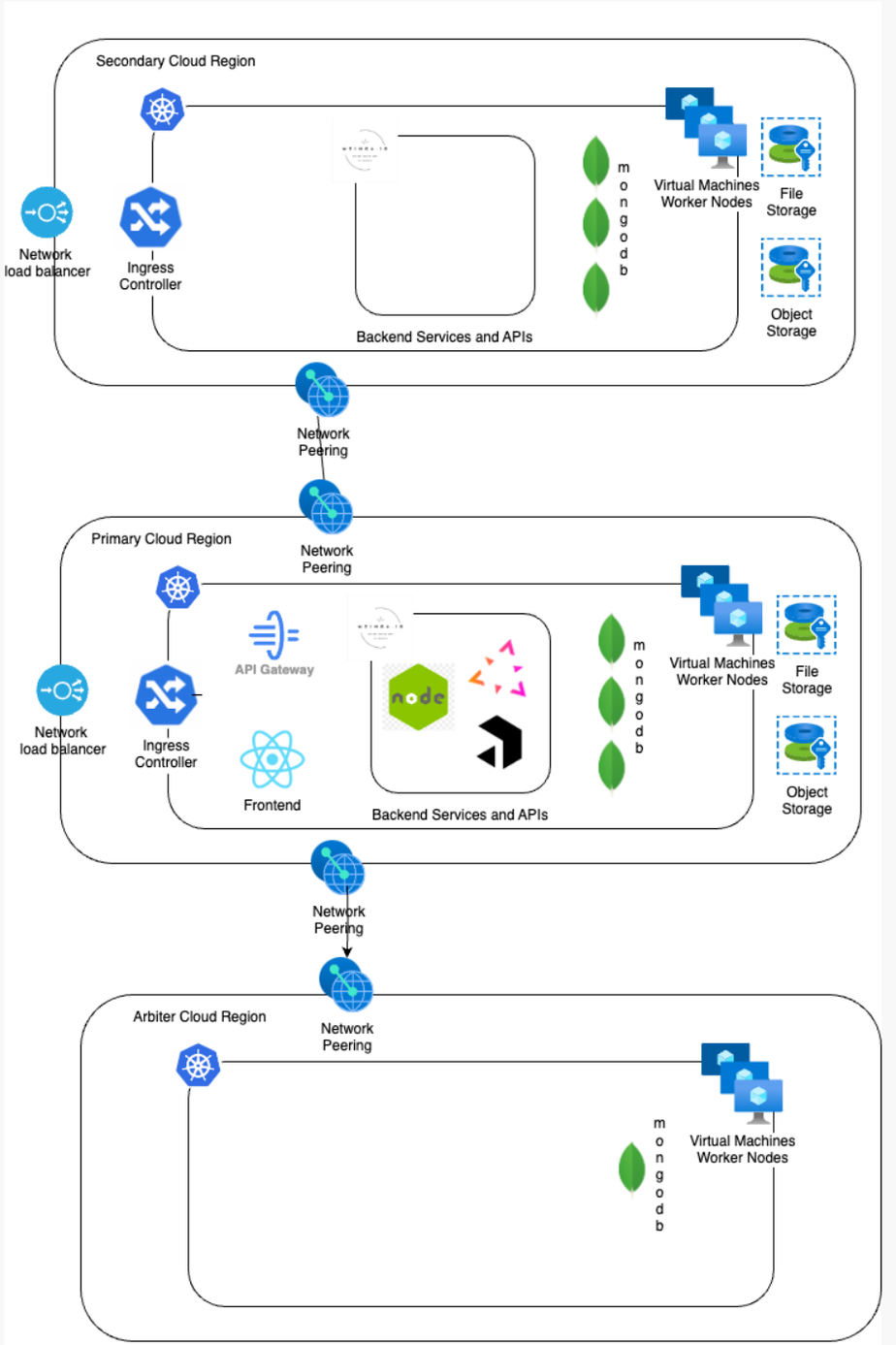
Final Solution

RPO close to zero

RTO below 5 minutes

No need for any manual intervention

4 B2MS machines and one Network Load Balancer (and some disk space) more compared to single region



#GlobalAzureTorino

Relevant Documentation

Keda (Kubernetes Event-driven Autoscaling) @ https://github.com/kedacore/keda

https://github.com/kedacore/http-add-on/tree/main

Follow us @:

https://globalazuretorino.welol.it/

https://www.meetup.com/it-IT/meetup-microsoft-azure-torino/


• BY COMMUNITY • FOR COMMUNITY •
#GLOBALAZURE
TORINO